UNIVERSIDAD TECNOLÓGICA
DE TAMAULIPAS NORTE

PRACTICA 7:

"EDIT CATEGORY."

TECNOLOGÍAS DE LA INFORMACIÓN

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

PRESENTA:

Luis Felipe Rivera Ortiz

REYNOSA, TAMAULIPAS **FEBRERO 2026.**

Índice

Tema **Página**

1.1. Controlador - Métodos de Edición 1
1.2. Vista de Edición (Edit.cshtml)..... 2
1.3. Sistema de Notificaciones con Toastr..... 3
1.4. Resultado Final..... 4

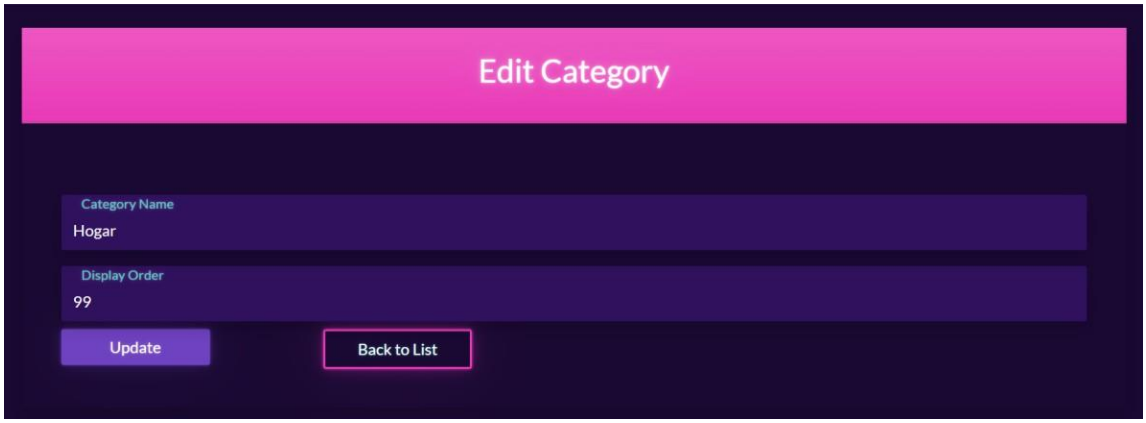
1.1. Controlador - Métodos de Edición.

Se implementaron los métodos Edit en el controlador para cumplir con la operación 'Update' del CRUD. El método GET recibe un identificador (Id), busca el registro correspondiente en la base de datos mediante Entity Framework (_db.Categories.Find(Id)) y lo envía a la vista. El método POST valida las modificaciones recibidas y, si son correctas, actualiza el registro en la base de datos (_db.Categories.Update(obj)), guardando los cambios y redireccionando al usuario.

```
// GET: Edit
0 referencias
public IActionResult Edit(int? Id)
{
    if (Id == null || Id == 0)
    {
        return NotFound();
    }

    // Buscar la categoría en la base de datos por su Id
    Category? categoryFromDb = _db.Categories.Find(Id);

    if (categoryFromDb == null)
    {
        return NotFound();
    }

    return View(categoryFromDb);
}

// POST: Edit
[HttpPost]
0 referencias
public IActionResult Edit(Category obj)
{
    if (ModelState.IsValid)
    {
        _db.Categories.Update(obj);
        _db.SaveChanges();
        TempData["success"] = "Category updated successfully";
        return RedirectToAction("Index");
    }
    return View(obj);
}
```

1.2. Vista de Edición (Edit.cshtml).

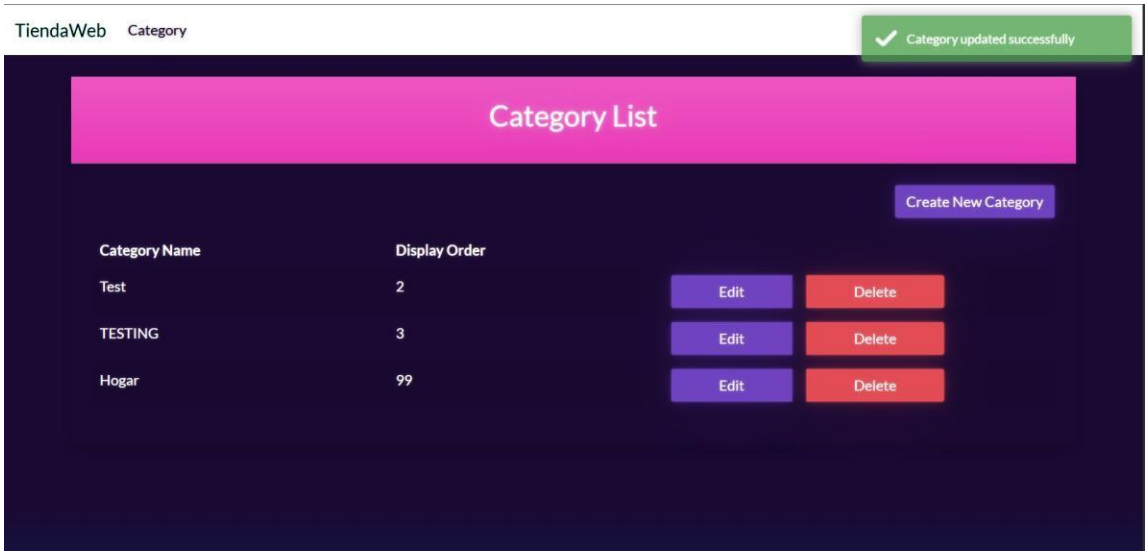
Se desarrolló la vista Edit.cshtml, reutilizando la estructura visual de la vista de creación pero adaptada para la actualización de datos. Mediante los Tag Helpers de ASP.NET Core, el formulario se precarga automáticamente con los valores actuales de la categoría seleccionada. Se incluyeron validaciones del lado del cliente referenciando el script _ValidationScriptsPartial.



1.3. Sistema de Notificaciones con Toastr.

Se creó una vista parcial llamada _Notification.cshtml dedicada a manejar las alertas del sistema. Se integró la librería de JavaScript **Toastr** para reemplazar las alertas estándar del navegador por notificaciones emergentes estilizadas. El código evalúa las variables de sesión temporales (TempData["success"] y TempData["error"]) y dispara el evento de notificación correspondiente.

```
@if (TempData["success"] != null)
{
    <script src="~/lib/jquery/dist/jquery.js"></script>
    <script src="//cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/js/toastr.min.js"></script>
    <script type="text/javascript">
        toastr.success('@TempData["success"]');
    </script>
}

@if (TempData["error"] != null)
{
    <script src="~/lib/jquery/dist/jquery.js"></script>
    <script src="//cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/js/toastr.min.js"></script>
    <script type="text/javascript">
        toastr.error('@TempData["error"]');
    </script>
}
```

1.4. Resultado Final.

Se verificó el funcionamiento integral del módulo. Al modificar una categoría y guardar los cambios, el sistema redirige al listado principal y despliega de manera exitosa la notificación emergente dinámica confirmando la actualización del registro en la base de datos.